SQL JOINs

Simple Guide - Step by Step

How to get data from two tables using one query

What this guide uses	
Tables	EMPLOYEES and DEPARTMENTS only
Database	Oracle HR schema
English level	A1 - very simple words
Goal	Understand what JOIN does, row by row

Part 1: Why Do We Need JOINs?

Think about this: you have two tables. The EMPLOYEES table has employee names and a number called department_id. The DEPARTMENTS table has department names and a number called department_id. But you want to see: employee name + department name in one result. The problem is: these two pieces of information are in two different tables. You can not get them with one simple SELECT.

A JOIN is the answer. A JOIN takes two tables and connects them using a common column. In our case, the common column is department_id. It exists in both tables. This column is like a bridge between the two tables. SQL uses this bridge to put the right data together.

> Simple idea: JOIN = take Table A + take Table B + find rows with the same value in the bridge column + put them together.

Our two tables (small example):

employee_id	first_name	department_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

department_id	department_name
10	Administration
20	Marketing
90	Executive
110	Accounting

The bridge is department_id. It is in both tables. Look at Steven: his department_id is 90. In the departments table, 90 = Executive. So Steven works in the Executive department. A JOIN does this for every row, automatically.

Part 2: INNER JOIN - Row by Row

INNER JOIN means: keep only rows that match in BOTH tables. If a row in one table has no match in the other table, SQL removes it.

The query:

```
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

How to read this: Take employees (call it e). Take departments (call it d). Connect them where e.department_id = d.department_id. Show first_name from e and department_name from d.

Now, what does SQL do inside? Row by row:

Step 1: Take first row of employees: Steven, department_id = 90
Look in departments: is there a row with department_id = 90? YES, it is "Executive".
Add to result: Steven | Executive

Step 2: Take next row: Neena, department_id = 90
Look in departments: is there a row with department_id = 90? YES, it is "Executive".
Add to result: Neena | Executive

Step 3: Take next row: Lex, department_id = NULL
Look in departments: is there a row with department_id = NULL? NO. NULL can not match anything.
Skip Lex. He is NOT in the result.

Step 4: Take next row: Diana, department_id = 10
Look in departments: is there a row with department_id = 10? YES, it is "Administration".
Add to result: Diana | Administration

Step 5: Take next row: Kimberely, department_id = 10
Look in departments: is there a row with department_id = 10? YES, it is "Administration".
Add to result: Kimberely | Administration

What about departments with no employees?
Marketing (20) and Accounting (110) have no employee with that department_id.
They are NOT in the result. INNER JOIN removes them too.

Final result:

first_name	department_name
Steven	Executive
Neena	Executive
Diana	Administration
Kimberely	Administration

Notice: We started with 5 employees + 4 departments = 9 rows total. But the result has only 4 rows. Lex is gone (NULL department). Marketing is gone (no employees). Accounting is

gone (no employees). INNER JOIN only keeps rows that MATCH in both tables.

Part 3: LEFT JOIN - Row by Row

LEFT JOIN means: keep ALL rows from the first table (the table after FROM). If a row has no match in the second table, write NULL for its columns. The first table is the LEFT table. That is why it is called LEFT JOIN.

Same tables, same query, just one word different:

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

What changes? Let me show you row by row:

Step 1: Steven, department_id = 90 -> Match found (Executive) -> Steven | Executive

Step 2: Neena, department_id = 90 -> Match found (Executive) -> Neena | Executive

Step 3: Lex, department_id = NULL -> No match in departments. BUT this is LEFT JOIN! Do NOT remove Lex. Keep him. Write NULL for department.
Lex | NULL

Step 4: Diana, department_id = 10 -> Match found (Administration) -> Diana | Administration

Step 5: Kimberely, department_id = 10 -> Match found (Administration) -> Kimberely | Administration

What about Marketing and Accounting? They are in the RIGHT table (departments). LEFT JOIN keeps the LEFT table (employees) safe. But it does NOT keep the RIGHT table. So Marketing and Accounting are still gone.

Final result:

first_name	department_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration

Compare with INNER JOIN: The only difference is Lex is back! He has NULL for department because he has no department_id. But LEFT JOIN does not throw him away. Rule: LEFT JOIN keeps ALL left table rows. No exceptions.

Part 4: RIGHT JOIN - Row by Row

RIGHT JOIN means: keep ALL rows from the second table (the table after JOIN). If a row has no match in the first table, write NULL for its columns. This is the opposite of LEFT JOIN.

```
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

Now SQL starts from the RIGHT table (departments). Row by row:

Step 1: Administration, department_id = 10 -> Look in employees: Diana(10) and Kimberely(10) match -> Diana | Administration AND Kimberely | Administration

Step 2: Marketing, department_id = 20 -> Look in employees: nobody has department_id = 20. But this is RIGHT JOIN! Do NOT remove it. Write NULL.
NULL | Marketing

Step 3: Executive, department_id = 90 -> Look in employees: Steven(90) and Neena(90) match -> Steven | Executive AND Neena | Executive

Step 4: Accounting, department_id = 110 -> Look in employees: nobody has department_id = 110. RIGHT JOIN keeps it. Write NULL.
NULL | Accounting

Final result:

first_name	department_name
Diana	Administration
Kimberely	Administration
NULL	Marketing
Steven	Executive
Neena	Executive
NULL	Accounting

Look what happened: Marketing and Accounting are in the result now (with NULL employee name). But Lex is gone again! Because Lex is in the LEFT table, and RIGHT JOIN protects the RIGHT table. Rule: RIGHT JOIN keeps ALL right table rows. No exceptions.

Part 5: FULL OUTER JOIN - Row by Row

FULL OUTER JOIN means: keep ALL rows from BOTH tables. No row is removed. If there is no match on one side, write NULL. This is the most complete JOIN. It combines LEFT and RIGHT together.

```
SELECT e.first_name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
ON e.department_id = d.department_id;
```

What changes? Nothing is removed!

- Steven matches Executive -> kept
- Neena matches Executive -> kept
- Lex has NULL department_id -> no match -> but FULL OUTER keeps him! Lex | NULL
- Diana matches Administration -> kept
- Kimberely matches Administration -> kept
- Marketing has no employee -> but FULL OUTER keeps it! NULL | Marketing
- Accounting has no employee -> but FULL OUTER keeps it! NULL | Accounting

Final result:

first_name	department_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration
NULL	Marketing
NULL	Accounting

Everyone is here! All 5 employees + all 4 departments. Lex and Marketing and Accounting are included with NULL where there is no match. Rule: FULL OUTER JOIN keeps everything. Nothing is lost.

Part 6: All 4 JOINs Compared

All 4 JOINs use the same two tables. Same data. Only the JOIN word is different. Look at the results side by side:

Row	INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER JOIN
Steven | Executive	YES	YES	YES	YES
Neena | Executive	YES	YES	YES	YES
Lex | NULL	NO	YES	NO	YES
Diana | Administration	YES	YES	YES	YES
Kimberely | Administration	YES	YES	YES	YES
NULL | Marketing	NO	NO	YES	YES
NULL | Accounting	NO	NO	YES	YES
Total rows in result	4	5	6	7

Simple rules to remember:

INNER JOIN: Only matching rows. Everything else is gone.

LEFT JOIN: ALL rows from first table (FROM). Second table: only matches.

RIGHT JOIN: ALL rows from second table (JOIN). First table: only matches.

FULL OUTER JOIN: ALL rows from both tables. Nothing is removed.

How to remember:

LEFT = protect the table on the LEFT (FROM table).
RIGHT = protect the table on the RIGHT (JOIN table).
INNER = protect nobody. Only matches survive.
FULL = protect everybody. Nothing is lost.

SQL template (copy and change one word):

```
SELECT e.first_name, d.department_name
FROM employees e
[INNER / LEFT / RIGHT / FULL OUTER] JOIN departments d
ON e.department_id = d.department_id;
```

The ON clause is the bridge. It tells SQL: connect rows where the values in these two columns are the same. Without ON, SQL can not connect anything. The column names do not need to be the same, but the values must match. In our example, both columns are called department_id and the values (10, 20, 90) must be the same in both tables.

When to use which JOIN?

You want ...	Use this JOIN
Only employees who have a department	INNER JOIN
ALL employees, even those without department	LEFT JOIN
ALL departments, even those without employees	RIGHT JOIN
ALL employees AND ALL departments	FULL OUTER JOIN
Find employees with no department	LEFT JOIN + WHERE dept_id IS NULL